EL-2003 – Computer Organization & Assembly Language Lab
Lab # 4

Objectives:

- Physical implementation of conditional and unconditional control transfer using JMP and conditional jump instructions
- Use of the CMP instruction and its effect on the FLAGS register
- Distinguishing signed vs. unsigned comparisons and choosing the correct jump instruction accordingly
- Applying conditional jumps to traverse and process arrays stored in memory, building on direct/indirect addressing from Lab 3

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. First think about statement problems and then write your program.
2. Do not copy from any source otherwise you will be penalized with negative marks.
3. Add your source code in word document and attach output screenshots.
4. Upload word file with codes and screenshot.
5. Submit your lab on Google Classroom.

Question 1:

Write and execute an ASM program that sets AX to 0, then uses JMP to unconditionally jump to a label named start, skipping over two instructions placed immediately after the jump: ADD AX, 1 followed by ADD AX, 2, which must never execute. At the start label, place ADD AX, 3 followed by ADD AX, 4, and terminate the program using MOV AX, 0x4C00 and INT 21h. The expected final value is AX = 7, since only the 3 and 4 get added while the 1 and 2 are skipped entirely. Attach the code, a screenshot of CMD compiling the .asm file to .com, and an AFD screenshot showing IP jumping directly to start along with the register panel confirming AX = 7 at the end.

Question 2:

Write and execute an ASM program that sets AX = 5 and BX = 5, compares them using CMP AX, BX, and uses JE to jump to a label named equal if they match. If the jump is not taken, set CX = 0 and jump to a label named done; if the jump is taken, go to the equal label, set CX = 1, and jump to done as well, terminating the program there with MOV AX, 0x4C00 and INT 21h. Since AX equals BX in this first run, the JE jump should be taken and CX should end up as 1. Then change the values to AX = 5 and BX = 8 and run the program again, confirming that CX now ends up as 0 since the values no longer match. Attach the code, the CMD compilation screenshot, and AFD screenshots of the FLAGS register immediately after CMP for both runs, clearly showing which flag changed between the equal and not-equal cases.

Question 3:

Load the value 125 into AL and compare it against -126 using CMP, then write two separate branches off this single comparison, one using JA, which treats the values as unsigned, and one using JG, which treats the values as signed, and show that the two branches can lead to different outcomes even though both are reacting to the exact same flags produced by the same CMP instruction. Attach the code, the CMD compilation screenshot, and AFD screenshots showing both branches executing, and include a short-written explanation of why the processor itself does not distinguish between signed and unsigned values, and why that interpretation is entirely up to which specific jump instruction the programmer chooses to use afterward.

Question 4:

Store ten numbers in memory using a dw list, then write a program that uses indirect addressing with BX as a pointer to walk through all ten values one at a time, adding each one to a running sum kept in AX, advancing BX by two bytes on every iteration since each value is a word, and using CMP together with JNE against a counter to determine when all ten values have been processed and the loop should stop. Once finished, store the final sum back into memory using direct addressing. Attach the code, the CMD compilation screenshot, and AFD screenshots showing the register state at the start of the traversal, partway through it, and after the final sum has been written to memory, along with a short note connecting how last week's addressing modes work made this week's traversal possible.